Parallel Programming: Practical Laboratory Project

Carles Carrillo, Edixon Parraga, Verònica Vidal and Pau Cascuboerta
Computer Architecture and Operating Systems
Universitat Autònoma de Barcelona
Cerdanyola, Spain
carles.carrillo@uab.cat, edixon.parraga@uab.cat, veronica.vidal@uab.cat, pau.casacuberta@uab.cat

***Resum*—The document presents the practice that we will do in the signing of the High Benefits Committee during the second semester of the 2026-2027 academic year. With the dual objective of putting into practice the knowledge obtained in this subject and that you acquire self-name work habits, the laboratory practices will focus on the development of a single project with different approaches.**

We present a set of possible milestones (implementing three parallel versions of the program's functional features that we provide you) and deadlines (deliverables) by which you will need to submit parts of the work you've completed. In this document, we also introduce Neural Networks as a solution strategy for certain types of problems, in particular, pattern recognition.

***Index Terms*—High-performance computing, practical, parallel applications, OpenMP, MPI, CUDA, neural networks**

I. I. INTRODUCTION

With the dual objective of putting into practice the knowledge gained in this course and of helping you develop habits of autonomous work, the lab sessions will focus on the development of an open-ended project for the parallelization of a real program. In this project, we present you with a set of possible milestones (implementing functional parallel versions of the program we provide) and deadlines (deliverables) by which you will have to submit parts of the work you've done. The planning necessary to achieve these milestones, and even deciding which milestones you want to pursue, is your responsibility.

In this document, in Section II, we introduce Neural Networks as a solution strategy for certain types of problems, in particular pattern recognition. Next, we present the most important features of the original code that we provide and that you will need to parallelize. Next, in Section III we describe the project's milestones—that is, the parallelizations we require—and offer some general recommendations. Finally, Section IV explains how the project will be evaluated and the deadlines for your deliverables.

II. Problem Description

The code we provide corresponds to the C implementation of a multi-layer perceptron neural network[1]. Neural networks, like the human nervous system, are composed of neurons (processing units) connected in what are known as layers, which operate simultaneously and interconnectively. A neural network is therefore a set of interconnected artificial neurons that uses a mathematical or computational model of data processing based on a connectionist approach to computation.

A. Perceptron-type Neural Networks

A Perceptron-type neural network consists of at least three layers of nodes:

- input layer
- hidden layer
- output layer

This type of neural network has, as shown in Figure 1, an input layer of neurons, each of which is connected to all the neurons of an intermediate layer known as the hidden layer. The neurons in this hidden layer can be connected to all the neurons in another hidden layer or to the neurons in an output layer.

The behavior of a neuron consists of integrating the values present at its inputs and deciding whether to fire or not based on the result.

To understand the internal workings of a neural network, you need to know that each neuron in each layer contains its weights, which indicate the neuron's importance (how a

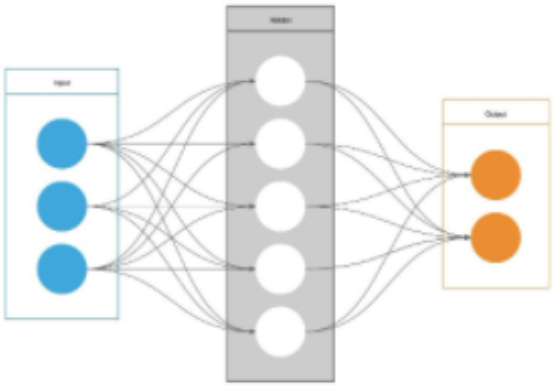


Figura 1. Three-layer perceptron..

[1]For more information, you can consult https://ca.wikipedia.org/wiki/Xarxa_neuronal

neuron in one layer influences all the neurons in the next layer). Finally, all the inputs in each layer are multiplied by their respective weights and summed.

This organization of simple processing elements is particularly well-suited for pattern classification and/or recognition.

In order to use a neural network, a training phase is necessary, in which it is presented with a set of inputs with a known output so that the network can adapt the connection weights between the neurons of different layers. Thus, in the code we provide, you will find the function `train_neural_net()` responsible for this phase.

Once trained, the network enters a usage phase, where it can receive inputs with an unknown output, which will be classified according to the values determined during the training phase. In the code we provide, you will find the function `test_nn()` that implements this phase.

B. Description of the provided code

In this project we provide you with the sequential C code that implements a three-layer perceptron (with one hidden layer) and an initial set of test data.

The code is an adaptation of what you can find on the page: https://www.geeksforgeeks.org/ml-neural-network-implementation-in-c-from-scratch/ (ML – Neural Network Implementation in C++ From Scratch).

The adaptation includes multiple aspects:

- **Initial and partial code optimization.** Memory access improvements.
- **Modularization of the code.** A set of modules is created to separate the different implemented functionalities.
- **Code extension** to work on the dataset selected for the project https://archive.ics.uci.edu/dataset/80/optical+recognition+of+handwritten+digits. Image reading routines and adaptation of data structures to the case at hand.

Basically, a network has been implemented that attempts to recognize images, such as those shown in Figure 2, which contain characters in the set '0'-'9'. Each image is the same size: 32×32 pixels (occupies 1 KByte in memory) and always shows the text as ones on a zero background.

The network parameters we provide you are as follows:

- **Number of training patterns**: 1934 (189 0s, 198 1s, 195 2s, 199 3s, 186 4s, 187 5s, 195 6s, 201 7s, 180 8s i 204 9s).
- **Number of patterns to recognize**: 946 (87 0s, 97 1s, 92 2s, 85 3s, 114 4s, 108 5s, 87 6s, 96 7s, 91 8s i 89 9s).
- **Number of patterns**: 10 (digits del 0-9).
- **Nombre de neurones d'entrada**: 1024 (cada un dels punts de la imatge: 32x32).
- **Number of neurons in the hidden layer**: We will have 3 hidden layers with 500, 210 and 135 neurons, corresponding (proper value determined by trial and error, don't think you will find many.
- **Number of output neurons**: 10 (one for each possible value).
- **Batch size**: 50 (number of patterns that will be processed per batch).

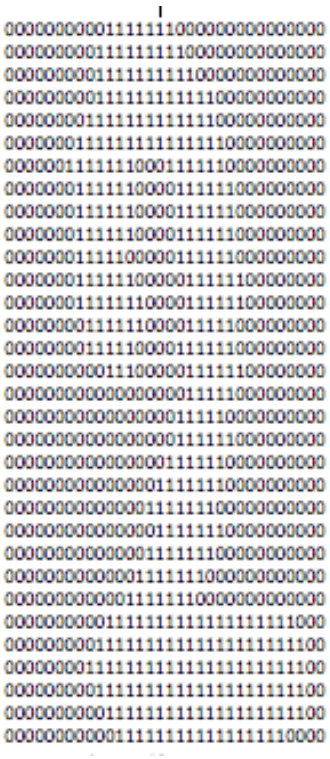


Figura 2. Example of a processed image.

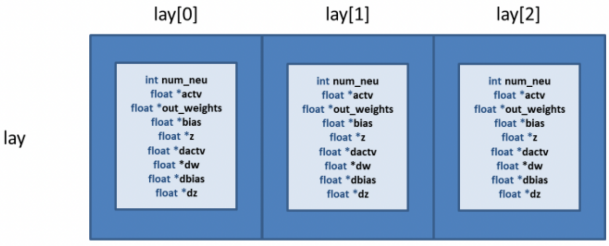


Figura 3. Data structure representing the XN.

To store represent the neural network, an array of layers is created where each layer is a structure that includes 8 arrays that represent each of the attributes of the neurons in the layer. The arrays of the attributes are for each neuron: actv (activation value), bias (value of the initial bias), z (value present at the input of the neuron [excitation]), dactv (change in the desired value of activation), dbias (change in the initial bias) and dz (change in the excitation value), all of these arrays have as many positions as neurons the layer has. The arrays corresponding to the weights are out weights (each row indicates how neurons in the current layer affect a neuron in the next layer) and dw (calculated change over weights), these arrays have (num neurons next layer x num neurons current layer) positions. This is because the weights refer to both the current and the next layer, and it is for this reason that the last layer does not need either of these last two arrays. Figure 3 shows this data structure.

Thus, the memory requirements of the application depend on:

- The number and size of patterns (training patterns in the training phase and recognition patterns in the recognition phase), since all patterns used in a phase are loaded into memory.
- The number of neurons in each layer. The input layer depends on the size of the pattern, the output layer of the set of patterns to be recognized, and the hidden layer of what is determined empirically..

From a computational point of view, the training function (`train_neural_net()`) is the most intensive, as it performs multiple passes through all training patterns, adjusting for each pattern the weights between the different layers of neurons.

However, in the real use of this type of optimization techniques, this phase is carried out infrequently, but for us it will be the one that will have the most weight.

The second most computationally intensive function is `test_nn()`. The complexity is less than the previous one because we no longer do iterations to adjust the weights, we only recognize the patterns that we give it from the start.

The code is extensible, as you can add more training and/or recognition patterns, as well as extend the set of work patterns (add encoded figures in the same way). To do this, you just need to modify the configuration file.

To compile the code, as provided, you need to use a C compiler (`gcc`) and add the mathematical library to use some of the functions of this library.

Finally, the list of elements provided with this statement is as follows:

- Commented source code of the neural network.
- File with the 1934 training patterns (`optdigits.tra`).
- File with the 946 recognition patterns (`optdigits.cv`).

III. Parallelization

The main objective of the project that we propose is to analyze the provided implementation of a perceptron-type neural network using different approaches: shared memory (multi core), distributed memory with message passage and shared memory with the use of accelerators (many core). This way you will see what differences there are between the implemented versions.

The second objective is to perform an analysis of the performance of the application. A behavioral analysis of the implemented versions will be carried out, they will be executed using different configurations, corresponding measures will be taken and one or more performance issues will be identified (indicating their potential importance). We will use the available analysis tools (`TAU, nsys nvprof, ncu, perf`, etc.).

A. Shared memory - multi core

We'll be using OpenMP to do this implementation.

In this case, we must first decide whether we are going to make a paralysis based on loops or on tasks.

If it is based on loops, it will be necessary to identify which loops can be parallelized, what data dependencies exist and look for solutions that introduce the least possible overhead.

If it is task-based, it will be necessary to define what it is a task in this program and possibly the code will have to be modified considerably.

B. Distributed memory - message step

We will use OpenMPI to make this implementation.

In this case, it is necessary to determine the parallelization criterion (data/functional) taking into account that in this approximation the grain must be thick to give more calculation to each node and, at the same time, reduce communications between nodes as much as possible.

Once this is done, it is necessary to identify the necessary communications and synchronizations, taking into account the overload involved.

NOTE: the file system is distributed and, therefore, each process can access the image files to load them into its memory. There is no need to copy them or send them over the network from one process to the next.

C. Shared memory with the use of accelerators - many core

We will use OpenACC to do this implementation. Implementation with CUDA is optional and allows you to add an extra point to the practice grade.

In this case, the most important thing is to identify the sections of code that can benefit from running on the accelerator and that, therefore, justify running a kernel on the GPU. In addition, it is necessary to clearly identify the operation that is carried out on the data with the aim of using algorithms known (and possibly well optimized) for accelerators.

D. Desenvolupament

Functional testing of your OpenMP, MPI, and OpenACC-CUDA programs will be done remotely. Remember to use the available execution queues, i.e. the `test.q, aolin.q, and cuda-ext.q` queues in the lab, or the `nododo.q and new-nodo.q` queues in the Wilma cluster.

The tests for the performance analysis of your OpenMP and MPI programs will be done on the Wilma cluster (queues `nododo.q and new-nodo.q`). As for the practice with OpenACC-CUDA you will have to use the `cuda-ext.q` queue of the architecture lab.

E. Recommendations

Here are some recommendations on how to develop the practice:

- You have at your disposal documents on the operation of the two available departmental clusters (laboratory cluster and Wilma cluster) and the submission of jobs to the resource manager, as well as a guide for using the TAU tool. Let them serve! And if you don't have enough, go to the original manuals.

- Some tests can be time-consuming, and in the case of performance analysis, they can generate very large amounts of information. Consequently, it may be a good idea to generate scaled-down versions of your program (by doing only a few iterations, for example).
- Use make, i.e. write Makefiles that allow you to generate different versions of the code (e.g. complete, reduced) and clean the working directory of executable files and object code, in an agile way.
- Organize the project appropriately in your user account. Make the commits locally and on github to have all the code updated. Add descriptive comments to your engagements.
- Comment on your code with helpful feedback (mostly by yourself). In order for a comment to be useful, it is very important that it is accompanied by the date and the author.

IV. Delivery and Evaluationó

The final delivery of the parallel implementation of the neural network and the corresponding documentation will be made to the Virtual Campus in the data indicated below. Two versions cannot be delivered on the same delivery date. Teachers reserve the right to request an interview so that group members can explain how their code works.

A final submission consists of:

- A functional implementation of the neural network that will be shown to the practical teachers (OpenMP, MPI, CUDA).
- A memory that describes:
  - the strategy of paral·lelitzation,
  - the changes made to the code,
  - performance measures: the acceleration (speed up) and efficiency obtained compared to the original using different numbers of resources,
  - the performance analysis performed,
  - the problem(s) encountered, its importance and possible solutions,
  - In addition, the memory will indicate in detail how the program should be compiled and executed.
- The report in **pdf** and the source code (`*.h, *.c, *.cu`) of each delivery will be automatically saved to Github on the date indicated below.

The practical grade represents the **35%** of the final grade of the subject. The value of each delivery has the weight in the final grade of the second subject shown in the table I.

Each delivery has an individual component that comes from, on the one hand, the follow-up in the classes (attendance, activity on Github, answers given to the teachers' questions during the deliveries) and on the other hand, from the validation test. The grade of each delivery will be applied to the factor defined according to the grade obtained from the validation test (Figure II). The validation tests will be carried out together with the deliveries on the dates indicated. It will be an individual test with questions about practice

Taula I
Weight of each delivery in the practical final grade.

OpenMP	**11,6%**
MPI	**11,6%**
OpenACC-CUDA	**11,6%**

Taula II
Factor applied according to the grade of the Validation Test.

Validation Test Grade	**Factor**
0.0-4.9	**0.5**
5.0-7.5	**0.75**
7.6-10.0	**1.0**

V. Conclusions

We have already presented the project that you will do during this semester. Any doubts or questions you have, please contact us `carles.carrillo@uab.cat`, `edixon.parraga@uab.cat`, `veronica.vidal@uab.cat` and `pau.casacuberta@uab.cat`. Do not hesitate to ask for tutorials!